PRÁCTICA 01
Mantenimiento de una aplicación heredada

Ingeniería de Software en un entorno empresarial simulado

NexaSoft Solutions | Mesa de ayuda técnica

Rol: desarrollador/a de mantenimiento. **Tecnología:** Flutter/Dart. **Modalidad:** equipos de 2 o 3, bitácora individual.
Duración sugerida: cuatro sesiones de trabajo. **Entradas:** proyecto Flutter y plantilla de bitácora en Word.

1. Situación laboral

Acabas de incorporarte a *NexaSoft Solutions*. Un cliente contrató mantenimiento de su aplicación de incidencias técnicas. La persona que construyó el sistema ya no trabaja en la empresa y la documentación es limitada. El proyecto inicia, muestra información y permite registrar incidencias; sin embargo, el cliente detectó fallos y solicitó dos mejoras.

Tu misión no es rehacer la aplicación. Debes investigar el código existente, reproducir los errores, proponer cambios acotados, probarlos y registrar evidencia verificable. El líder técnico (profesor) dará seguimiento mediante tickets. Se permite investigar documentación y usar IA conforme a la política de confidencialidad indicada en esta guía.

2. Objetivos de aprendizaje

- Explorar y comprender software construido por otras personas.
- Identificar causas de defectos lógicos y de persistencia sin depender de instrucciones que señalen el archivo exacto.
- Extender funcionalidades sin introducir regresiones.
- Documentar hipótesis, pruebas, cambios, uso de IA y decisiones profesionales.
- Organizar el trabajo mediante Git y comunicar avances técnicos de forma clara.

3. Preparación del proyecto

1. Descomprime el ZIP del proyecto. Confirma que contiene `pubspec.yaml`, `lib/` y `assets/`.
2. En la terminal, ubicada en la raíz del proyecto, ejecuta:

```
flutter create . --project-name nexasoft_incidencias --platforms android,web,windows
flutter pub get
flutter devices
flutter run -d chrome
```

3. Si utilizas Android, puedes cambiar `chrome` por el identificador de tu dispositivo. En Windows, `flutter run -d windows` requiere herramientas de compilación instaladas. Las carpetas nativas se generan localmente; **no borres ni reemplaces `lib/` ni `assets/`**.

4. Recorre la aplicación, abre detalles, cambia estados, registra una incidencia y prueba el buscador. Toma evidencias *antes de modificar el código*.

5. Crea un repositorio Git local si el profesor no entregó uno. Registra el estado inicial y trabaja en una rama por ticket cuando sea posible.

Importante: La aplicación inicia con un JSON ficticio, pero luego emplea almacenamiento local. El botón de *Restablecer demostración* borra los cambios locales y regresa a los datos iniciales. Úsalo solo para reiniciar escenarios de prueba y anótalo en la bitácora.

4. Tickets del cliente

Cada ticket debe documentarse con: pasos para reproducir, comportamiento actual, comportamiento esperado, hipótesis, evidencia antes/después, archivos modificados, commit y pruebas de regresión. Un ticket no está cerrado únicamente porque "parece funcionar".

BUG-001 — El cambio de estado desaparece

Reporte: Un operador modifica una incidencia de *Pendiente* a *Resuelta*. Se ve actualizada, pero al cerrar completamente y volver a abrir la aplicación reaparece el estado anterior.
Criterios de aceptación:

- Cambiar el estado desde el detalle y comprobarlo en la lista.
- Cerrar y abrir de nuevo la aplicación; el nuevo estado debe conservarse.
- Registrar una incidencia nueva y verificar que sigue existiendo tras reiniciar.
- El cambio de estado de un registro no debe alterar los demás.

Evidencia mínima: mismo ID antes del cambio, después del cambio y tras reiniciar; explicación de la causa, no solo captura final.

BUG-002 — La búsqueda omite registros existentes

Reporte: El cliente escribe `wifi` y no obtiene el caso titulado "WiFi intermitente en recepción". También necesita buscar palabras que aparecen únicamente en la descripción.
Criterios de aceptación:

- La búsqueda debe ignorar diferencias entre mayúsculas y minúsculas.
- Debe considerar tanto el título como la descripción.
- Ignorar espacios sobrantes al inicio y al final de la consulta.
- Si no hay texto, mostrar los registros que correspondan a los filtros activos, sin descartar entradas.

Pruebas sugeridas: buscar `wifi`, `WiFi` y `túnel` (término incluido en la descripción de la incidencia sobre VPN).

BUG-003 — El tablero presenta cantidades incorrectas

Reporte: Después de cambiar el estado de una incidencia, el número de pendientes mostrado en la pantalla principal a veces deja de coincidir con la lista.
Criterios de aceptación:

- Los totales por estado deben coincidir con los datos actuales de la aplicación.
- Verificar al pasar *Pendiente* a *Resuelta* y viceversa.
- Verificar el recuento después de registrar un caso nuevo y después de reiniciar la aplicación.
- Los totales generales no deben depender de la búsqueda o de los filtros visuales.

Evidencia mínima: captura y conteo manual antes/después, más explicación del criterio utilizado para calcular los totales.

FEAT-001 — Clasificación por prioridad

Solicitud: Cada incidencia debe tener una prioridad *Baja*, *Media* o *Alta*.
Criterios de aceptación:

- Permitir elegir prioridad al crear una incidencia.
- Mostrar la prioridad tanto en la lista como en el detalle y permitir modificarla desde el detalle.
- Conservar la prioridad tras cerrar y volver a abrir la aplicación.
- Las incidencias antiguas que no tienen prioridad asignada deben abrirse sin fallar y utilizar *Media* como valor inicial.
- Evitar valores distintos de los tres aprobados.

Evidencia mínima: nuevo registro con prioridad Alta, cambio de prioridad y persistencia tras reiniciar.

FEAT-002 — Filtros combinables

Solicitud: La pantalla principal debe permitir filtrar por estado y prioridad, además de la búsqueda por texto.
Criterios de aceptación:

- Filtro de estado: *Todos*, *Pendiente*, *En proceso*, *Resuelta*.
- Filtro de prioridad: *Todas*, *Baja*, *Media*, *Alta*.
- Texto, estado y prioridad deben funcionar juntos, aplicando todas las condiciones activas.
- Debe existir una forma visible de restablecer la consulta y ambos filtros.
- Ante cero resultados, mostrar un mensaje claro, no una pantalla rota.

Pruebas sugeridas: buscar una palabra, seleccionar Pendiente y Alta; luego limpiar y confirmar que reaparecen todos los registros.

5. Organización del trabajo por sesiones

Sesión	Trabajo principal	Evidencias esperadas
Día 1	Clonar/preparar, explorar código, dibujar un mapa simple de sus componentes y reproducir tres fallos.	Capturas del comportamiento original, notas de investigación, primer commit.
Día 2	Investigar y corregir BUG-001, BUG-002 y BUG-003 por prioridad acordada con el líder.	Pruebas antes/después, commits y explicación de las causas.
Día 3	Implementar FEAT-001; adaptar lectura de registros anteriores y verificar guardado.	Evidencia visual, escenarios con datos nuevos y antiguos, commits.
Día 4	Implementar FEAT-002, hacer pruebas combinadas y entregar una versión revisada.	Pruebas de regresión, reporte final de tickets, bitácora y repositorio.

La distribución es una planeación, no una obligación de concluir determinado ticket en un día. Si un error requiere más investigación, documenta qué se intentó, qué se aprendió y cuál es el siguiente paso. **No ocultes bloqueos ni inventes resultados.**

6. Uso de Git y revisión

- Un integrante crea una rama clara por ticket, por ejemplo `fix/bug-002` o `feat/prioridades`; el equipo coordina cambios y evita sobrescribir trabajo.
- Cada commit debe explicar la modificación concreta. Evitar mensajes como “cosas”, “ya quedó” o “final”.
- Antes de integrar, otro integrante revisa al menos una prueba y busca regresiones.
- Registrar en Word el identificador corto del commit y la prueba relacionada. Si no cuentan con remoto, el trabajo Git local sigue siendo válido.

7. Política simulada de confidencialidad e IA

Regla de la empresa: está permitido utilizar herramientas de IA *autorizadas* y documentación pública. No se puede enviar código de la empresa, capturas internas, bases de datos, registros de clientes, contraseñas, claves, tokens ni configuraciones confidenciales a un servicio externo *no autorizado*. Aunque aquí los datos son ficticios, debes actuar como si el sistema fuera real.

Consultas permitidas: explicar un concepto general (p. ej., persistencia en Flutter), pedir estrategias abstractas de depuración o solicitar un ejemplo mínimo inventado sin copiar código interno. También se puede usar una herramienta corporativa aprobada conforme a sus reglas, si existiera autorización expresa.

Consultas no permitidas: pegar archivos completos del repositorio, enviar trazas sin revisar que no contienen datos, subir pantallas de clientes, proporcionar secretos o usar un asistente que indexe automáticamente todo el proyecto sin autorización.

Ejemplo de pregunta segura: "En una aplicación Flutter con almacenamiento local, al cambiar una propiedad en memoria se ve actualizada pero no se conserva al reiniciar. ¿Qué puntos genéricos debería comprobar para diagnosticar la persistencia? Da un ejemplo ficticio mínimo."

Antes de consultar IA: clasifica la información, elimina identificadores y datos internos, revisa la configuración y permisos del asistente y solicita autorización si hay duda. **Después:** verifica cada sugerencia con tus propias pruebas. Nunca afirmes que la IA resolvió el error si no puedes explicarlo.

8. Cómo llenar la bitácora diaria en Word

Entrega **un archivo Word por alumno y por jornada**, utilizando la plantilla `Bitacora_Diaria_NexaSoft.docx`. Puedes duplicar la plantilla diariamente. El archivo se puede guardar como `Apellido_Nombre_Dia01.docx` y así sucesivamente.

1. **Encabezado:** coloca nombre, grupo, fecha, sesión, equipo y tickets atendidos. Indica el tiempo aproximado de trabajo real.
2. **Actividad y problema:** anota qué intentabas lograr, los pasos exactos para reproducir el defecto y qué resultado observaste. Evita "no funciona" sin detalles.
3. **Investigación y cambios:** escribe hipótesis, pruebas fallidas, decisión tomada, archivos revisados/modificados y commit. Distingue *investigación* de *solución implementada*.
4. **Evidencias:** inserta imágenes legibles *dentro de las áreas señaladas* o agrega páginas si es necesario. Cada captura debe tener ID (E1, E2...), descripción, ticket, paso que prueba y fecha. Incluye resultado previo, posterior y, cuando corresponda, persistencia tras reiniciar. Puedes usar capturas de la app, consola, diff o historial Git; no adjuntes datos privados.
5. **Pruebas:** registra entradas, resultado esperado, resultado obtenido y si pasó o falló. Un problema pendiente también es un resultado válido si está bien explicado.
6. **Uso de IA:** marca Sí o No. Si la usaste, identifica herramienta, objetivo, resumen o fragmento *sanitizado* del prompt, tipo de datos compartidos, cómo protegiste la confidencialidad, qué sugerencia aceptaste/rechazaste y prueba de verificación. No es necesario pegar la conversación íntegra.
7. **Cierre:** señala qué está terminado, qué queda bloqueado y el primer paso previsto para la jornada siguiente.

Ejemplo breve de evidencia bien escrita: "E2 – BUG-002 – tras buscar 'wifi', la lista muestra INC-102. Prueba en Chrome; antes: 0 resultados, después: 1 resultado. Commit: a1b2c3d. No se compartió código interno con IA."

9. Entregables y evaluación

1. Proyecto actualizado y repositorio con historial de cambios (o ZIP con carpeta `.git` si así lo indica el profesor).
2. Bitácora individual de **cada día**, con evidencias incrustadas en el Word.
3. Registro de tickets: abierto, en proceso, resuelto o bloqueado, y la prueba que respalda cada estado.
4. Resumen final del equipo: qué funcionó, qué sigue pendiente y riesgos conocidos.

Criterio	Valor
Diagnóstico y reproducción de los defectos	20 %
Correcciones y pruebas de regresión (3 BUG)	25 %
Funcionalidades nuevas y persistencia (2 FEAT)	25 %
Bitácoras, evidencias y seguimiento técnico	20 %
Git, revisión y uso responsable de IA	10 %
Total	**100 %**

Criterio profesional: una corrección sin pruebas documentadas no se considera cierre completo. Un problema no resuelto pero investigado rigurosamente genera evidencia válida de aprendizaje; simular capturas, inventar pruebas o exponer información confidencial no es aceptable.